



Human-in-the- Loop UX – Khi Nào Agent Cần Xin Phép?

AICB-P2T3 · Ngày 27 · Chương 6 – Agent trong Production

Giảng viên

HÃY SUY NGHĨ...

“Agent tự quyết hay hỏi người dùng – ranh giới nào là an toàn và không làm phiền?”

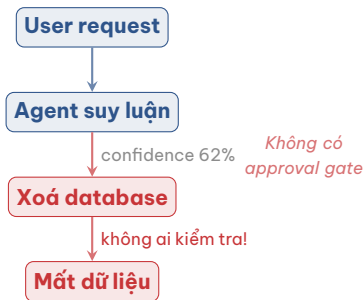
Giữ câu hỏi này trong đầu khi học bài hôm nay

1. Tại sao Full Autonomy nguy hiểm?
2. HITL Taxonomy – 6 Interaction Patterns
3. Confidence Routing – Khi nào interrupt?
4. Approval Workflows & Implementation
5. 2026 Product Landscape
6. Feedback Loops & Audit Trails
7. HITL UX Best Practices

Tại sao Full Autonomy nguy hiểm?

Từ sự cố thực tế đến nhu cầu Human-in-the-Loop

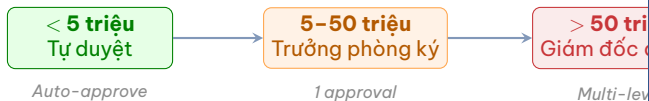
01



Bài học từ agent products 2025–2026:

- Read/search/debug dần được **auto** vì reversible và dễ audit
- Write/delete/deploy vẫn cần **human checkpoint** hoặc sandbox boundary
- External side effects (email, Slack, Git push, prod API) là nơi products hiện đại dừng lại để hỏi

Lưu ý: Xu hướng mới không phải “agent hỏi mọi thứ” mà là **bounded autonomy**: auto bên trong ranh giới đã định, hỏi khi



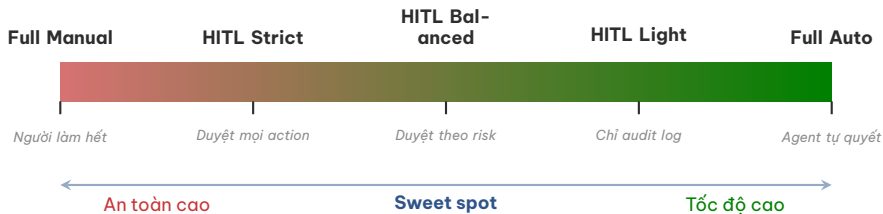
Dưới threshold → agent tự xử lý
Vùng trung gian → cần 1 lần duyệt

Rủi ro cao → phải escalate lên người có thẩm quyền

Cùng nguyên tắc: chi phí sai lầm quyết định mức kiểm soát

HITL Taxonomy – 6 Interaction Patterns

Phân loại cách con người tham gia vào quyết định của agent



Không có vị trí “đúng” cố định – sweet spot phụ thuộc vào **domain risk**, **agent maturity**, và **user tolerance**. Bắt đầu strict, nới dần khi trust được xây dựng.

#	Pattern	Khi nào?	Ví dụ
1	Approval	Side effect / policy boundary	Deploy, xóa data, gửi email
2	Clarification	Input mơ hồ	“Report Q1 hay Q2?”
3	Structured Elicitation	Thiếu field bắt buộc	Form chọn env, repo, approver
4	Review Checkpoint	Output cần kiểm	Draft email, code PR, SQL patc
5	Edit / Correction	Human muốn chỉnh	Sửa args trước khi chạy tool
6	Escalation	Vượt thẩm quyền	Pháp lý, tài chính, security

Read-only = auto

Write in sandbox = auto hoặc re-view

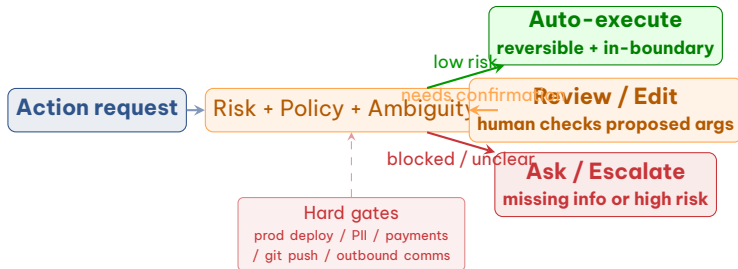
External / destructive = manda-

Approval UX tốt không chỉ có **Approve / Reject**.

Nó phải hỗ trợ **edit args**, **fill form**, hoặc **respond as human tool**.

Confidence Routing – Khi nào interrupt?

Agent tự đánh giá confidence để quyết định hỏi hay tự làm



Lưu ý: Update quan trọng: **confidence không đủ**. Products mới ưu tiên **policy-aware gating**: dù model rất tự tin, action vẫn phải dừng nếu chạm dữ liệu nhạy cảm, network, hoặc side effect bên ngoài.

Reversible

Có thể undo?
Nếu có, ưu
tiên auto

External

Có side effect?
Nếu có, ưu
tiên review

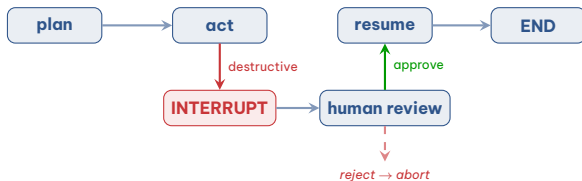
Unknown

Thiếu field?
Nếu có, hỏi
bằng form

Thay vì chỉ tune một số confidence, hãy đo 4 biến: **reversibility**, **blast radius**, **policy sensitivity**, và **missing information**. Trend mới là **interrupt by action class**, không phải chỉ **interrupt by score**.

Approval Workflows & Implementation

LangGraph interrupt/resume và Streamlit approval UI



2026 LangGraph mindset:

- Pause tại **tool call** hoặc **custom node** khi chạm policy
- Human có thể **approve / edit / reject / respond**
- Resume phải dùng **same thread/checkpoint** để state bền vững

1. Surface interrupt payload + re-view config
2. Human chọn decision cho từng action
3. `Command(resume=...)` đưa quýt

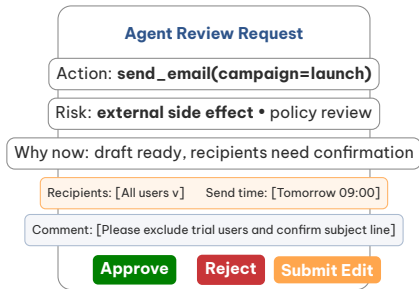
```
from langchain.agents import create_agent
from langchain.agents.middleware import import
    HumanInTheLoopMiddleware
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import Command

agent = create_agent(
    model="gpt-5.5",
    tools=[write_file, send_email, ask_user],
    middleware=[HumanInTheLoopMiddleware(
        interrupt_on={
            "write_file": {"allowed_decisions": ["approve"
            , "edit", "reject"]},
            "send_email": {"allowed_decisions": ["approve"
            , "edit", "reject"]},
            "ask_user": {"allowed_decisions": ["respond"
            ]},
        }
    )],
    checkpointer=InMemorySaver(),
)
```

3 cập nhật quan trọng:

1. **Policy per tool:** tool nào cho approve/edit/reject/respond
2. **Checkpoint bắt buộc:** pause phải survive refresh / reconnect
3. **Respond** dành cho “ask human” tool, không dùng để từ chối action nguy hiểm

Lưu ý: Nếu edit args quá mạnh, model có thể re-plan lại. Vì vậy approval UI nên khuyến khích **small edits** thay vì viết lại toàn bộ action.



Approval UX đang dịch chuyển sang:

1. **Review card:** what / why / risk / rollback
2. **Editable fields:** dropdown, text box, date/time
3. **Scoped approvals:** once / session / specific destination
4. **Form-first elicitation:** hỏi đúng field còn thiếu thay vì “Bạn có đồng ý không?”

Codex đã có
tool/requestUserInput.
Claude Code có permission modes

2026 Product Landscape

OpenAI, Anthropic, Google, Cursor – các agent lớn đang làm Human-in-the-Loop như thế nào?

4 lực đẩy chính:

1. Agent không còn chỉ **trả lời** mà đã **thực thi**
2. Enterprise muốn autonomy nhưng không chấp nhận “YOLO production”
3. Mobile / remote control làm approval **không cần ngồi cạnh máy**
4. Compliance team đòi **audit trail + named approver**

2024: hỏi rất nhiều, dễ phiên
2025: bắt đầu phân biệt read vs write
2026: hỏi ít hơn nhưng hỏi ở đúng ranh giới

Từ khóa: **bounded autonomy**

Lưu ý: HITL mới không phải “human babysitting”. Nó là **control plane** cho các action có blast radius cao.

Từ docs hiện tại của Codex:

- **Sandbox mode** xác định agent **có thể** làm gì
- **Approval policy** xác định agent **khi nào phải dừng lại để hỏi**
- App / MCP tools có side effects cũng có thể trigger approval
- Có **granular approval policy** thay vì một mode duy nhất

Codex tách rõ
permission boundary
khỏi
model reasoning quality

Trong docs/changelog đến **June 25, 2026**, Codex còn hỗ trợ **auto-review** cho approval requests đủ điều kiện thay vì luôn bắt user bấm tay.

2 pattern mới đáng chú ý:

1. `tool/requestUserInput`:
agent có thể hỏi bằng câu hỏi có header, lựa chọn, và timeout
2. **Codex Remote GA** (June 25, 2026):
review progress và approve actions từ **điện thoại**

Human input đang chuyển từ
modal yes/no
sang
small structured interaction

Lưu ý: Đây là shift rất lớn: HITL không còn bị khóa trong IDE/terminal. Approval có thể trở thành **async workflow**.

Current CLI/docs expose rõ các mode:

- default
- acceptEdits
- plan
- auto
- dontAsk
- bypassPermissions

Claude Code biến mức autonomy thành **user-facing mode**, không phải config ẩn.

Điều này giúp team dạy user: khi nào chỉ plan, khi nào cho edit, khi nào cho auto.

Lưu ý: HITL tốt không chỉ là prompt đẹp. Nó phải có **mode model** để user hiểu hệ thống đang ở mức rủi ro nào.

Điểm rất mạnh trong current docs:

- **PreToolUse**: allow / deny / ask / defer
- **PermissionRequest**: auto-handle prompt nếu match hẹp
- **updatedInput**: hook có thể chỉnh input trước khi chạy
- **defer**: pause tool call, hỏi user qua UI khác, rồi resume

Claude Code cho phép HITL sống ở **policy layer**, không chỉ trong model prompt.

Đây là hướng rất enterprise-friendly.

Lưu ý: Nhưng nếu matcher quá rộng, hook có thể vô tình auto-approve quá nhiều. Bài học: luôn **scope hẹp**.

Theo docs hiện tại của Gemini Computer Use:

- Model có thể trả về `safety_decision.require_confirmation`
- App phải surface prompt xin phép rồi gửi lại `safety_acknowledgement`
- Có **prompt injection detection** ở mức screenshot
- Hỗ trợ browser / mobile / desktop environments

Điền hết form trước rồi chỉ hỏi ở **final irreversible step** như Send / Submit / Confirm Purchase

Gemini push rất mạnh idea: **ask at the last responsible moment**. Đây là pattern đáng đưa vào mọi agent UX.

2 hướng HITL nổi bật của Cursor trong 2026:

1. **Plan Mode:** research codebase, hỏi làm rõ, xuất plan có thể review trước khi code
2. **Auto-review Run Mode** (May 29, 2026):
allowlisted calls chạy ngay,
sandboxed calls chạy trong sandbox,
còn lại qua classifier subagent

Cursor tối ưu cho coding workflow:

review-before-write

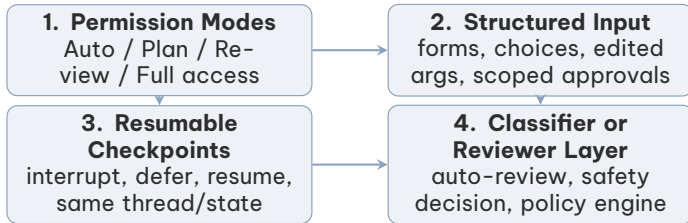
và

classifier-before-risky-exec

Lưu ý: Điểm mới là human không phải review mọi step. Hệ thống tự cố gắng classify trước để giảm số lần interrupt.

Vendor	Boundary	Human input UX	Special thing	Risk model
Codex	sandbox + approval policy	approval prompt, remote approval, requestUserInput	tách permissions khỏi elicitation	boundary-first
Claude Code	permission modes + hooks	ask / allow / defer / resume	programmable approval layer	policy-first
Gemini	safety decision + system instruction	require_confirmation + acknowledge	ask after prep, before final click	action-class-first
Cursor	plan + classifier + sandbox	plan review + auto-review + approval	giảm approval fatigue bằng classifier	classifier-first

Các product lớn đang hội tụ về một pattern: **bounded autonomy + resumable checkpoints + structured human input**. Khác biệt chính nằm ở **ai quyết định gate** và **user thấy gate** đó dưới dạng nào.



Không vendor nào còn tin vào một threshold confidence duy nhất. Họ đang xây agent quanh **modes, policies, checkpoints, và reviewer layers**.

Anti-pattern 1: Yes/No mọi thứ

- User mệt
- Approve cho xong
- Log không giúp học được gì

Anti-pattern 2: Hỏi quá sớm

- Chưa thấy diff
- Chưa điền đủ field
- Human chưa đủ context để quyết

Anti-pattern 3: Chỉ dựa vào confidence

- Model rất tự tin vẫn có thể click nhầm
- Không encode policy/legal boundaries

Anti-pattern 4: Không có edit path

- Reject xong agent phải làm lại từ đầu
- Human không sửa được một field nhỏ

Lưu ý: Nếu approval UI không cho human **edit**, **scope**, hoặc **defer**, hệ thống sẽ vừa chậm vừa bực.

Checklist của enterprise buyer:

- Ai phê duyệt action nào?
- Có **scoped approvals** không?
- Có replay / audit log / reviewer identity không?
- Có quản trị permission mode bằng admin policy không?
- Có chặn data exfiltration / legal acceptances / purchases không?

HITL đang trở thành **governance surface** chứ không chỉ là UI pattern.

Nếu muốn deck này cập nhật, hãy dạy sinh viên map **action classes** trước rồi mới vẽ approval UI.

1. **Bắt đầu bằng action classes:** read, write-in-sandbox, external, destructive
2. **Thiết kế 3 loại human input:** approve, edit args, fill structured form
3. **Pause đúng lúc:** sau khi prepare xong, trước final irreversible step
4. **Persist state:** interrupt phải resume cùng thread/checkpoint
5. **Đo sau khi ship:** approval rate, latency, regret, rollback

Big tech không cố bỏ human hoàn toàn. Họ đang **nén human attention** vào vài thời điểm có giá trị nhất.

Feedback Loops & Audit Trails

Thu thập phản hồi, ghi log, và tăng autonomy dần

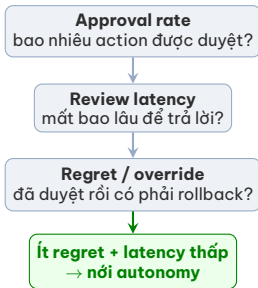
```
class AuditEntry(BaseModel):
    timestamp: datetime
    agent_id: str
    action: str
    confidence: float
    risk_level: str # low/med/high
    reviewer_id: str | None
    decision: str # auto/approve/reject
    reason: str | None
    execution_time_ms: int
```

```
# Immutable: append-only PostgreSQL
# Backup: S3 daily snapshot
# Replay: reconstruct full session
```

Mỗi event ghi:

- **Who:** agent nào, reviewer nào
- **What:** action gì, confidence bao nhiêu
- **When:** timestamp chính xác
- **Why:** reasoning của agent + decision

Lưu ý: Audit trail là **compliance requirement** (GDPR, SOC2). Build from day one – **cannot retrofit** sau khi đã production.



3 metrics mới nên theo dõi:

1. **Approval rate**: quá thấp → agent đang xin phép sai chỗ
2. **Review latency**: cao → form/UI chưa đủ rõ hoặc hỏi quá nhiều
3. **Regret rate**: human approve xong vẫn rollback → gating chưa đủ tốt

Chỉ tăng autonomy khi **approval cao và regret thấp**.

Nếu approval cao nhưng regret cũng cao, nghĩa là reviewer đang approve “cho xong”.

Đo trust bằng hành vi, không đo

HITL UX Best Practices

Thiết kế trải nghiệm không làm phiền user

07

Thiếu input thì hỏi **sớm**.
Side effect thì hỏi **muộn**, sát lúc thực thi.

Nếu field đã biết cấu trúc, dùng dropdown / checkbox / date picker.
Không nên bắt user gõ lại cả câu.

Chỉ hiện **what / why / risk / diff / rollback**.
Đừng bắt user đọc transcript dài mới dám bấm.

Giảm cognitive load để human chỉ dùng attention cho chỗ thật sự quan trọng.

Hỗ trợ **approve once, approve this session**, hoặc **approve for this destination**.

Đây là cách giảm fatigue mà vẫn giữ boundary.

Log decision, latency, regret, roll-back.

Mỗi interrupt phải giúp policy tốt hơn ở lần sau.

Lưu ý: Nếu một approval screen không cho edit, scope, hoặc defer, nó thường sẽ tạo ra ma sát thay vì tạo trust.

Một HITL screen tốt phải giúp user quyết nhanh hơn, không phải khiến user làm lại công việc của agent.

Demo & Thực hành

HITL Code Review Agent + Lab hands-on

08

1. Agent đọc PR, phân loại action: comment-only, edit patch, push branch, gọi external tool
2. Low-risk: auto draft review comments; medium-risk: hiển thị diff + suggested patch để reviewer edit
3. High-risk: push branch / trigger CI / post external comment → approval card hoặc structured form
4. Mỗi decision lưu thành audit event để đo latency, regret, và các chỗ agent hỏi quá nhiều

Mục tiêu: Build HITL agent với LangGraph interrupt + Streamlit approval UI

Deliverable: HITL agent + approval UI + confidence-based routing + PostgreSQL audit trail

Thời gian: 2 giờ

1. **Policy map**: phân loại tool nào auto, tool nào review, tool nào chỉ được hỏi bằng form
2. **LangGraph HITL**: approve/edit/reject/respond + checkpoint
3. **Approval UI**: review card + editable args + one field dạng dropdown/text input
4. **Analytics**: log approval rate, review latency, regret rate thay vì chỉ log confidence

A/B test 2 UXs: modal yes/no vs structured form – đo completion time, approval quality, và regret rate trên 20 test cases

Những ý chính cần nhớ trước khi sang bài tiếp theo

- 1 Xu hướng mới là **bounded autonomy**: agent tự làm trong boundary, hỏi khi chạm risk / policy / ambiguity
- 2 HITL hiện đại không chỉ có approve/reject – còn có **edit, respond**, và **structured form input**
- 3 Audit trail + decision analytics là bắt buộc nếu muốn tăng autonomy mà vẫn giữ compliance
- 4 Progressive autonomy chỉ nên tăng khi **approval cao** và **regret thấp**

Ngày 28: Workshop Tổng Hợp – Full Production Agent System

“Tất cả components đã build xong – N28 là ngày ghép lại thành hệ thống production hoàn chỉnh”

- Hoàn thành Lab 27: HITL agent + audit trail
- Review lại tất cả labs N16–N27 – chuẩn bị integration

Hỏi & Đáp

HITL có làm chậm agent không? Khi nào nên bỏ approval gate?



Cảm ơn!

AICB-P2T3 · Ngày 27 · Human-in-the-Loop UX

`github.com/vinuni-aicb`

Liên hệ: `instructor@vinuni.edu.vn`